Answerspot — Root Tooling & Health Module

Makes the commands every other artifact references actually resolve.

`package.json` (repo root — npm workspaces)

json

```
{
  "name": "answerspot",
  "version": "0.1.0",
  "private": true,
  "workspaces": [
    "apps/web",
    "apps/api",
    "packages/*"
  ],
  "engines": {
    "node": ">=20"
  },
  "scripts": {
    "lint": "npm run lint --workspaces --if-present",
    "typecheck": "npm run typecheck --workspaces --if-present",
    "test": "npm run test --workspaces --if-present",
    "build": "npm run build --workspaces --if-present",
    "dev:web": "npm run dev --workspace=apps/web",
    "dev:api": "npm run start:dev --workspace=apps/api",
    "prisma:generate": "prisma generate --schema apps/api/prisma/schema.prisma",
    "prisma:migrate": "prisma migrate dev --schema apps/api/prisma/schema.prisma",
    "prisma:deploy": "prisma migrate deploy --schema apps/api/prisma/schema.prisma",
    "prisma:seed": "node apps/api/prisma/seed.js"
  },
  "devDependencies": {
    "prisma": "^5.18.0",
    "typescript": "^5.5.0",
    "eslint": "^9.8.0",
    "prettier": "^3.3.0"
  }
}
```

`Makefile` (repo root)

makefile

```makefile
# Answerspot — dev shortcuts. Run `make help` for the list.
.DEFAULT_GOAL := help
.PHONY: help dev down migrate seed test lint logs build clean

help: ## Show this help
	@grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) \
	    | awk 'BEGIN {FS = ":.*?## "}; {printf "  \033[36m%-10s\033[0m %s\n", $$1, $$

dev: ## Start all services (web, api, workers, db, redis, minio)
	docker compose up --build

down: ## Stop all services
	docker compose down

migrate: ## Apply Prisma migrations (against running db)
	docker compose exec api npm run prisma:deploy

seed: ## Seed reference + demo data
	docker compose exec api npm run prisma:seed

test: ## Run all tests (node workspaces + python workers)
	npm run test
	cd apps/workers && pytest -q

lint: ## Lint + typecheck everything
	npm run lint
	npm run typecheck
	cd apps/workers && ruff check . && mypy answerspot_workers

logs: ## Tail all container logs
	docker compose logs -f

build: ## Build all production images locally
	docker compose build

clean: ## Stop and remove volumes (DESTRUCTIVE)
	docker compose down -v
```

apps/api/src/health/health.module.ts

typescript

```typescript
import { Module } from '@nestjs/common';
import { HealthController } from './health.controller';
import { PrismaService } from '../prisma/prisma.service';

@Module({
  controllers: [HealthController],
  providers: [PrismaService],
})
export class HealthModule {}
```

---

apps/api/src/health/health.controller.ts

typescript

```
import { Controller, Get } from '@nestjs/common';
import { ApiTags } from '@nestjs/swagger';
import { PrismaService } from '../prisma/prisma.service';

@ApiTags('health')
@Controller('health')
export class HealthController {
  constructor(private prisma: PrismaService) {}

  // Liveness: process is up. Used by K8s livenessProbe + CI smoke test.
  @Get()
  liveness() {
    return { status: 'ok', ts: new Date().toISOString() };
  }

  // Readiness: dependencies reachable. Used by K8s readinessProbe.
  @Get('ready')
  async readiness() {
    try {
      await this.prisma.$queryRaw`SELECT 1`;
      return { status: 'ready', db: 'up' };
    } catch {
      return { status: 'not-ready', db: 'down' };
    }
  }
}
```

> Register `HealthModule` in `app.module.ts` imports. Resolves to `/api/v1/health` (matches the probes in the Helm chart and the CI smoke tests).

`apps/api/prisma/seed.js`

javascript

```
// Seeds reference platforms + a demo org/business. Idempotent (upserts).
// Run: npm run prisma:seed
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

async function main() {
  const platforms = [
    { key: 'CHATGPT', displayName: 'ChatGPT' },
    { key: 'PERPLEXITY', displayName: 'Perplexity' },
    { key: 'GEMINI', displayName: 'Gemini' },
    { key: 'AI_OVERVIEW', displayName: 'Google AI Overview', enabled: false }, // ADR
  ];
  for (const p of platforms) {
    await prisma.platform.upsert({
      where: { key: p.key },
      update: { displayName: p.displayName, enabled: p.enabled ?? true },
      create: p,
    });
  }

  const org = await prisma.organization.upsert({
    where: { stripeCustomerId: 'demo' },
    update: {},
    create: { name: 'Demo Roofing Co', plan: 'PRO', status: 'TRIALING', stripeCustome
  });

  await prisma.business.create({
    data: {
      organizationId: org.id,
      name: 'Demo Roofing Co',
      category: 'roofer',
      city: 'Austin',
      state: 'TX',
      website: 'https://example.com',
    },
  });

  console.log('Seed complete.');
}

main()
```

```
  .catch((e) => { console.error(e); process.exit(1); })
  .finally(() => prisma.$disconnect());
```